MDPI

*Article*

# Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

**Kun Li [1,*], Ankur Lohachab [1], Michel Dumontier [1] and Visara Urovi [1]**

[1] Department of Advanced Computing Sciences, Maastricht University, 6200 MD Maastricht, The Netherlands; ankur.lohachab@maastrichtuniversity.nl (A.L.); michel.dumontier@maastrichtuniversity.nl (M.D.); v.urovi@maastrichtuniversity.nl (V.U.)

* Correspondence: k.li@maastrichtuniversity.nl

**Abstract**

In blockchain-based healthcare data sharing, consent policies are expected to be enforced in a verifiable and auditable manner. However, public blockchain transparency may expose sensitive policy information through on-chain representations, verification interfaces, and observable execution artifacts. Existing privacy-preserving solutions mainly protect data contents, user identities, or transaction details, but they insufficiently address the confidentiality of governing consent policies and policy-dependent leakage during enforcement. To address this gap, we propose the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI), which enables authorization decisions to be verified while limiting leakage about policy content, policy identity, and policy structure and parameters. Policy confidentiality is conditional on an explicit off-chain trust boundary: the TPS must keep plaintext policy information and requester authorization evidence secret. To reduce policy-dependent leakage through public verification artifacts, TPS-ZKPI combines fixed-length policy encoding, a reusable zero-knowledge policy interpreter, commitment-based policy binding, and public-chain transcript normalization. We evaluate TPS-ZKPI against metadata-only adversaries that observe public-chain artifacts and conduct inference attacks on these three leakage targets. The results show that no modeled supervised or unsupervised attack recovers hidden policy information reproducibly above chance under a multi-seed leakage criterion, and a paired gas-padding ablation attributes the gas-channel protection specifically to randomized gas padding, at a gas overhead of about 38% on access and 40% on commit.

**Keywords:** blockchain; policy privacy; data use ontology; healthcare data sharing; zero-knowledge proof

## 1. Introduction

Blockchain technology has emerged as a promising foundation for decentralized and auditable healthcare data sharing [1–3]. Its transparency and immutability enable verifiable consent enforcement and traceable authorization decisions, supporting cross-institutional accountability. At the same time, consent enforcement on public blockchains may expose policy-dependent contextual information, even when the underlying health data remain encrypted [4,5].

In healthcare data sharing, a *consent policy* encodes the conditions under which a data subject authorizes others to access or use their health data, such as who may access the data, for what purpose, and under what constraints. Consent policies in healthcare are often privacy-sensitive [6]. For example, a policy based on biomedical data-use terms may

Received:
Accepted:
Published:

**Copyright:** © 2026 by the authors. Submitted to *Blockchains* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

reveal whether a dataset is restricted to disease-specific research, whether ethics approval is required, or whether access is limited to particular institutions or projects. If such policies are directly represented in smart contracts, embedded in verification logic, or reflected in public execution traces, unauthorized observers may infer information about the protected resource, the data subject's constraints, or the requester's authorization context.

Existing privacy-preserving blockchain solutions mainly protect data contents, user identities, or transaction details [7–9]. Encrypted data-sharing mechanisms such as attribute-based encryption and proxy re-encryption protect access to data [10,11], while zero-knowledge transaction systems hide asset ownership, values, or participant relationships [12]. None of these mechanisms protects the confidentiality of the policies that govern access decisions. Even mechanisms designed to hide policy content, such as hidden-policy attribute-based encryption [13], do not control the policy-dependent artifacts that a public chain exposes while enforcing the policy. In blockchain-based consent management, policy leakage may arise not only from explicit policy disclosure, but also from policy-dependent artifacts such as calldata size, public-input layout, emitted events, verifier interfaces, and gas usage. Therefore, policy-private consent enforcement requires more than hiding data or proving correctness in zero knowledge; it also requires controlling what the public-chain transcript reveals about the hidden policy. This paper studies this problem for blockchain-based healthcare data sharing under public-chain observability. The central challenge is to enable an on-chain verifier to check that a requester satisfies a provider-defined consent policy while ensuring that metadata-only public-chain observers learn minimal information about the governing policy and the requester's authorization evidence from the resulting verification transcript. We focus on Boolean-based consent policies, including policies derived from biomedical data-use conditions such as terms from the Data Use Ontology (DUO), the GA4GH standard vocabulary of consent-based data-use conditions [6], and disease-scope constraints [14].

We address this problem with the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI). In TPS-ZKPI, the data provider (who can be seen as the data owner) commits to a hidden consent policy on-chain and provisions the corresponding policy-bound proving material to a Trusted Proof Service (TPS). The requester submits policy-relevant authorization evidence, such as attributes, credentials, and access context, to the TPS. The TPS constructs the private witness and generates a zero-knowledge proof showing that the requester satisfies the committed policy. The proof is then verified on-chain against the provider's policy commitment. This yields an auditable authorization outcome without exposing the policy logic to the public-chain transcript.

TPS-ZKPI combines the following design elements. First, it uses TPS-assisted proof generation to keep confidentiality-sensitive policy material and requester evidence outside the public-chain transcript. Second, it compiles heterogeneous Boolean-based consent policies into fixed-length Reverse Polish Notation (RPN) programs. Third, building on the standard commit-and-prove paradigm, it evaluates these programs through a reusable zero-knowledge policy interpreter that avoids policy-specific verifier contracts and supports policy evolution without changing the public verification interface. Fourth, it applies observability-aware transcript normalization to reduce policy-dependent variation in calldata, logs, public parameters, and gas-related behavior.

We evaluate TPS-ZKPI using attack-driven empirical analysis under a metadata-only threat model. The evaluation operationalizes policy privacy through three objectives: Policy Content Hiding (PCH), Policy Indistinguishability (PI), and Structure and Parameter Privacy (SPP). Supervised and unsupervised attacks test whether observable public-chain artifacts allow adversaries to recover hidden policy content, distinguish otherwise-equivalent policies, or infer structural and parameter properties. The results show that,

after transcript normalization, no modeled attack recovers hidden policy information reproducibly above chance under a multi-seed leakage criterion. A paired ablation serves as a positive control: when gas padding is removed, the same attackers recover policy-dependent signal, and that signal collapses under the full mechanism, isolating randomized gas padding as the defense responsible for closing the gas channel.

This paper makes the following contributions:

- We propose TPS-ZKPI, a TPS-assisted zero-knowledge approach for publicly verifiable and auditable consent enforcement, in which on-chain proof verification and auditability hold without trusting the TPS, while policy confidentiality and faithful witness construction are conditional on an explicit, delimited trust assumption—keeping policy-bound proving information and requester authorization evidence off-chain.
- We develop a reusable zero-knowledge policy interpreter for fixed-length Boolean RPN policy programs over a bounded token alphabet, combining policy commitment binding with a policy-independent on-chain verification interface that serves all policies through one circuit and supports policy evolution without changing the on-chain verifier.
- We provide an attack-driven empirical evaluation of residual policy-dependent leakage from normalized public-chain metadata on the Polygon Amoy testnet[1], using supervised classification and unsupervised clustering attacks under a multi-seed permutation-test criterion. A paired gas-padding ablation then isolates randomized gas padding as the defense responsible for closing the gas channel and quantifies its gas-cost overhead.

The remainder of this paper is organized as follows. Section 2 reviews the technical foundations: cryptographic access control in blockchain healthcare, zero-knowledge proofs, and the Data Use Ontology. Section 3 discusses related work and positions TPS-ZKPI against existing blockchain consent-management, privacy-preserving blockchain, hidden-policy, private policy-evaluation, reusable proving, and metadata-leakage approaches. Section 4 defines the system and trust model, the public-chain adversary, and the privacy objectives. Section 5 presents the TPS-ZKPI design. Section 6 reports the empirical evaluation. Section 7 discusses limitations and future work, and Section 8 concludes the paper.

## 2. Background

This section reviews the technical foundations relevant to policy-private consent enforcement on blockchains. We first introduce the cryptographic access-control mechanisms commonly used in blockchain-based healthcare data sharing. We then review zero-knowledge proofs and the Data Use Ontology (DUO) [15], which together provide the basis for the policy representation and verification mechanisms used in our approach.

### *2.1. Cryptographic Access Control in Blockchain Healthcare*

Fine-grained consent management in blockchain-based healthcare data sharing is commonly built on cryptographic access-control primitives, chiefly Ciphertext-Policy Attribute-Based Encryption (CP-ABE) and Proxy Re-Encryption (PRE) [11,16,17]. In CP-ABE, a data owner embeds an attribute-based access policy into the ciphertext, so that only requesters whose credentials satisfy the policy can decrypt the protected data [10]. PRE lets a semi-trusted intermediary transform a ciphertext encrypted for one party into one decryptable by a designated requester, delegating access without exposing the plain-

[1] https://amoy.polygonscan.com

text [17]. These primitives establish who may obtain the data; how they relate to *policy* confidentiality on a public chain is discussed in Section 3.

### 2.2. Zero-Knowledge Proofs

Zero-knowledge proofs (ZKPs) are cryptographic protocols that allow a prover to convince a verifier that a statement is true without revealing any information beyond the validity of the statement [18]. Modern ZKP systems support verifiable computation over private inputs, so that complex predicates can be proven without disclosing the underlying data. In blockchain settings, ZKPs are widely used to reconcile public verifiability with privacy: privacy-focused systems such as Zerocash and Tornado Cash apply them to conceal transaction details or provide transaction anonymity [9,19]. These properties make ZKPs a natural building block for proving that a requester satisfies a hidden consent policy, which is the direction our approach develops.

### 2.3. The Data Use Ontology

The Data Use Ontology (DUO) provides a standardized vocabulary for expressing biomedical data-sharing consent conditions, and its terms are widely adopted in genomics and biobanking to enable semantic interoperability [6,15]. A DUO-based policy combines permissions and modifiers—for example, general research use (GR), disease-specific research (DS), or a requirement that ethics-committee approval be obtained (IRB)—into the condition under which access is granted.

#### DUO Term Categories

Although DUO offers a rich set of permissions and modifiers, its terms are heterogeneous in how they must be evaluated for compliance. To structure their handling in a zero-knowledge setting, we group DUO terms into four categories:

- **Boolean-only:** simple yes/no conditions that can be represented directly without external validation, e.g. “no restriction” (NRES), general research use (GR), or “health/medical/biomedical research” (HMB).
- **Ontology-linked:** terms whose evaluation involves consistency checks against external vocabularies or classifications, e.g. disease-specific research (DS), population origins or ancestry research (POA), or clinical care use (CC).
- **Obligation-required:** terms that impose procedural or temporal obligations, e.g. “publication required” (PUB), “publication moratorium” (MOR), “time limit on use” (TS), or “return to database” (RTN).
- **Credential-required:** terms whose evaluation relies on attested proof of requester status or approvals, e.g. “collaboration required” (COL), “ethics approval required” (IRB), “project-specific restriction” (PS), or “institution-specific restriction” (IS).

Table 1 summarizes representative DUO terms under this classification. Some terms reduce to simple Boolean flags, while others rely on ontology references, external obligations, or credential attestations. Later sections describe how these heterogeneous requirements are harmonized into a unified privacy-preserving verification process.

**Table 1.** Representative DUO terms grouped by evaluation category.

| Category | Example Terms | Description |
|---|---|---|
| Boolean-only | NRES (DUO:0000004), GR (DUO:0000042), HMB (DUO:0000006) | Direct yes/no conditions, no external validation required. |
| Ontology-linked | DS (DUO:0000007), POA (DUO:0000011), CC (DUO:0000043) | Evaluated against external ontologies such as ICD-10 or MONDO. |
| Obligation-required | PUB (DUO:0000019), MOR (DUO:0000024), TS (DUO:0000025), RTN (DUO:0000029) | Impose obligations such as publication duties, time limits, or data return. |
| Credential-required | COL (DUO:0000020), IRB (DUO:0000021), PS (DUO:0000027), IS (DUO:0000028) | Depend on credentials, approvals, or institutional/project-specific restrictions. |

## 3. Related Work

Blockchain-based consent management for healthcare data sharing has been studied extensively, with smart contracts used to record, enforce, and audit consent across institutions [1–3]. This line of work focuses on making consent decisions verifiable and auditable, and the privacy mechanisms surveyed in this space mainly protect health data contents and user identities [5]. The confidentiality of the governing consent policy—and what public-chain enforcement artifacts reveal about it—is typically out of scope.

Beyond the consent-management setting, existing work addresses important fragments of policy-private consent enforcement, but not their combination under public-chain observability. Encrypted data-sharing mechanisms, such as Ciphertext-Policy Attribute-Based Encryption (CP-ABE) and proxy re-encryption (PRE) [16,17], control access to encrypted data. However, standard CP-ABE attaches the access policy to the ciphertext in the clear, exposing the policy itself, while PRE-based delegation may reveal authorization patterns that data encryption is not designed to hide. Transaction-privacy systems, including Zerocash-style protocols [19] and Ethereum-oriented systems such as ZETH and AZTEC [7,8], provide public verification over private transaction witnesses, but they mainly target asset ownership, values, or participant relationships rather than compliance with a provider-defined consent policy.

Hidden-policy access-control work reduces direct policy exposure by using encrypted policies, hidden-policy ABE variants, off-chain matching, or policy-holder components [13,20]. Functional encryption gives a more general cryptographic abstraction for revealing function outputs while hiding inputs [21], but it does not by itself yield publicly verifiable authorization evidence. Private-state blockchain systems, including Secret Network and Oasis/Ekiden, protect smart-contract state or execution through encrypted state and trusted execution environments [22,23]. These lines of work address adjacent forms of policy or state hiding, but TPS-ZKPI targets a more specific setting: hiding consent policies on a public chain while retaining public verifiability of each authorization decision. Even where policies are hidden, existing designs may still leak policy-dependent artifacts through ciphertext structures, policy-update transactions, authorization workflows, or repeated interactions. Moreover, many target permissioned or off-chain settings, where enforcement artifacts are not globally visible as they are on public blockchains.

Privacy-preserving access-control and policy-evaluation mechanisms have been studied in credential-based and XACML/SAML settings [24], as well as in multi-party policy

evaluation using homomorphic encryption and secure function evaluation [25]. These approaches support confidential authorization computation, but they are not primarily designed for settings where policy-evaluation artifacts—calldata, public inputs, emitted events, timing, and gas usage—are permanently and globally visible on a public blockchain. In addition, these approaches do not directly provide compact evidence that a smart contract can verify against a committed, provider-defined policy instance.

Reusable proving systems, including universal-setup SNARKs, composable proof systems, and zkVMs [26–28], improve maintainability by reducing the need for policy-specific circuits or verifier contracts. However, when deployed for public-chain consent enforcement, reusable proving alone does not guarantee policy privacy: policy-dependent program representations, public inputs, calldata layouts, logs, or gas profiles may still remain observable. This motivates TPS-ZKPI's use of a domain-specific reusable policy interpreter, which preserves proof reuse while exposing only a policy-independent public verification interface.

Public-chain metadata-leakage studies show that cryptographic privacy guarantees may be weakened by observable transaction-level artifacts such as timing and volume patterns [4]. Techniques such as batching, decoys, and traffic shaping can reduce some observable signals [29], but they are usually designed for payment privacy, anonymity sets, or transaction-linkability settings. They do not directly provide hidden-policy compliance verification or auditable consent enforcement. In contrast, TPS-ZKPI treats metadata leakage as part of the policy-privacy objective rather than as a separate transaction-level concern.

Existing approaches address verifiable consent recording, data confidentiality, transaction privacy, policy hiding, confidential policy evaluation, proof reuse, or metadata mitigation in isolation, whereas policy-private consent enforcement under public-chain observability requires the combination of policy confidentiality, metadata-leakage control, verifiable auditability, and a stable verification interface for evolving Boolean-based consent policies. Table 2 summarizes this positioning.

**Table 2.** Positioning of representative approaches against policy-private consent enforcement under public-chain observability. "Yes" indicates explicit support, "Partial" indicates indirect or incomplete support, and "No" indicates that the dimension is not addressed—typically because it lies outside the approach's original design goals; assumptions in parentheses are part of the claim.

| Approach | Policy confidentiality | Metadata-leakage mitigation | Verifiable auditability | Main limitation |
|---|---|---|---|---|
| Blockchain consent management [1,2] | No | No | Yes | Records and enforces consent verifiably on chain, but policy confidentiality and policy-dependent leakage are typically out of scope. |
| Data / transaction privacy [7,8,16,17,19] | No | Partial | Partial | Protects encrypted data access or transaction semantics, but not the governing consent policy or policy-dependent authorization workflow. |
| Hidden-policy access control [13,20] | Partial | Partial | Partial | Reduces direct policy exposure, but may leak through workflows, updates, or policy-holder interactions. |
| MPC / HE policy evaluation [24,25] | Yes | No | Partial | Supports confidential computation, but usually lacks public-chain-verifiable authorization evidence. |
| Universal SNARKs / zkVMs [26–28] | No | Partial | Yes | Supports proof reuse and public verification, but does not by itself hide policy-dependent public artifacts. |
| Metadata-leakage defenses [29] | No | Yes | No | Models or mitigates observable signals, but does not provide hidden-policy consent enforcement. |
| TPS-ZKPI | Yes (under TPS trust) | Yes (modeled channels) | Yes | Provides policy-private, metadata-aware, verifiable consent enforcement under the stated TPS trust assumption. |

## 4. Problem and Threat Model

### *4.1. System and Trust Model*

We consider a blockchain-based healthcare data-sharing system in which two types of users interact to share data: 1. data providers, who provide protected resources and govern access to them through consent policies, and 2. data requesters, who seek access to these resources subject to those policies. TPS-ZKPI mediates this interaction through three components: a TPS operating off-chain, an on-chain access-control manager (ACM), and a data access layer. The provider defines a consent policy $P$ for a protected resource on behalf of the data subject whose consent it encodes, and registers an on-chain commitment $h$ to the encoded policy. The requester seeks access by submitting policy-relevant authorization evidence, such as attributes, credentials, and access context. The TPS constructs the private witness and generates a zero-knowledge proof of policy compliance. The ACM verifies the proof on-chain and records the authorization outcome, while the data access layer grants access according to the recorded decision.

TPS-ZKPI relies on the following assumptions. First, policies are provider-controlled: the provider is responsible for defining the applicable consent policy and committing the corresponding policy instance on-chain. Second, the TPS is trusted to preserve the confidentiality of policy-bound proving material and requester-submitted authorization evidence, and to follow the prescribed proof-generation workflow. Third, the commitment scheme is assumed to be hiding and binding, and the zero-knowledge proof system is assumed to satisfy completeness, soundness, and zero knowledge with respect to the implemented policy-compliance relation. Credential issuance, revocation, and real-world identity verification are treated as external functions.

Conceptually, the TPS is an independent proof service, distinct from the provider; this separation is what makes relocating trust meaningful, since it prevents the provider from being the single party that holds both the policy and the requester evidence.

Possible instantiations of such an independent service include a third-party proof service, a TEE-backed proof service that makes the trust assumption hardware-attested, or MPC/collaborative SNARK proof generation in which no single party sees both policy material and requester evidence.

For implementation and evaluation simplicity, the current evaluation co-locates the TPS with the provider (a provider-operated TPS). This co-location concentrates trust at the provider side: a single party now holds both the policy and the requester evidence, so a compromised provider-operated TPS could construct witnesses from substituted attribute values or withhold proof generation from legitimate requesters. Two guarantees must be distinguished here. Authorization correctness is primarily enforced by the commitment and on-chain proof verification: a proof that does not satisfy the committed policy relation is rejected, largely independently of how the TPS is instantiated. Policy confidentiality, in contrast, is not cryptographically enforced against the TPS itself; it rests entirely on the trust assumption stated above. Our evaluation therefore assumes a TPS that follows the prescribed workflow and isolates the remaining question: what a public-chain observer can infer from the resulting public artifacts. Weakening the TPS trust assumption itself is orthogonal and is the role of the reduced-trust instantiations above.

### *4.2. Public-Chain Adversary*

The primary adversary is a passive public-chain observer. The adversary can inspect smart-contract code and monitor public blockchain artifacts across multiple authorization sessions, including policy commitments, submitted proofs, public inputs, emitted events, calldata, timing, and gas usage. The adversary does not observe provider-to-TPS policy material, commitment-opening randomness, requester-submitted authorization evidence, credential witnesses, or TPS-side witness construction: these items are exchanged over off-chain channels or held locally by the provider and the TPS, and never appear in on-chain artifacts. We assume the off-chain channels themselves are confidential (e.g., TLS-protected).

Each policy-enforcement instance corresponds to an authorization session, in which a requester seeks access to a protected resource under a committed policy. For each such session, the adversary observes a public view

$$view_{\mathsf{pub}} = (h, \pi, id_{\mathrm{req}}, id_{\mathrm{res}}, nonce, pp, meta), \tag{1}$$

where $h$ is the on-chain commitment to the hidden policy, $\pi$ is the zero-knowledge proof submitted for on-chain verification, $id_{\mathrm{req}}$, $id_{\mathrm{res}}$, and *nonce* are the public requester, resource, and session-binding values, *pp* denotes other fixed-format public parameters or public handles, and *meta* denotes transaction-level metadata exposed by the enforcement layer, such as calldata footprint, event structure, public-input layout, timing, and gas usage. The adversary's goal is to infer hidden information about the committed policy or the authorization interaction from this public view.

In addition to the public-chain observer, we treat the requester as untrusted with respect to policy confidentiality. The requester initiates access requests, observes its own submitted evidence and authorization outcome, and inspects the public transcript. However, the requester does not receive the encoded policy, commitment-opening randomness, or policy-bound proving material, which remain within the provider-to-TPS proof-generation boundary. This restriction matters because a requester that learned the policy could infer sensitive facts about the data subject from the consent terms themselves and, in addition, would learn the policy's full acceptance region—which attribute combinations are granted—enabling targeted evidence construction or probing beyond its own legitimate sessions.

Compromised TPS behavior, adaptive requester probing, and off-chain side channels are non-goals for the current model. The SNARK checks internal consistency of the witness it is given; it does not prove that the private attribute vector is the requester's real-world attribute set. Binding an authorization to $id_{\text{req}}$—in the current implementation, at the transaction level rather than inside the proof statement—ties the recorded outcome to an identifier and session context, but without an issuer-verification layer it does not by itself bind every private attribute value to an external source of truth.

### *4.3. Privacy Objectives*

We operationalize policy privacy as three measurable, attack-oriented objectives that cover the information a public-chain observer could extract about a hidden policy: its semantic content, its identity relative to other policies, and its structural form. These correspond to Policy Content Hiding, Policy Indistinguishability, and Structure and Parameter Privacy, respectively.

*Policy Content Hiding (PCH).* PCH requires that public-chain artifacts should not reveal the semantic content of the hidden consent policy, such as data-use terms, disease categories, research purposes, institutional restrictions, or ethics-related requirements.

*Policy Indistinguishability (PI).* PI requires that different candidate policies should not become distinguishable from their public verification artifacts when they are compiled into the same fixed public interface. This captures whether two policies can be told apart from commitments, proofs, public parameters, or metadata even when their direct representations are hidden—that is, whether the identity of the governing policy can be narrowed down.

*Structure and Parameter Privacy (SPP).* SPP requires that public-chain artifacts should not reveal structural or parametric properties of the hidden policy, such as logical structure, nesting depth, predicate count, set size, or threshold-like parameters.

### *4.4. Problem Statement*

Given a provider-defined consent policy $P$, requester authorization evidence, a globally observable public blockchain, and the stated TPS trust assumption, the problem is to enforce access to a protected healthcare resource while satisfying three requirements. First, authorization must be publicly verifiable: the ACM should be able to verify that the requester satisfies the committed policy without learning or executing the policy in plaintext. Second, policy-dependent information should remain hidden from public-chain observers, including policy content, policy identity, structural properties, requester evidence, and metadata-level enforcement fingerprints. Third, authorization outcomes should remain auditable, so that successful access decisions can be recorded and checked without exposing the hidden policy logic.

## 5. TPS-ZKPI Design

### *5.1. Overview*

As shown in Figure 1, the TPS-ZKPI workflow binds a provider-committed hidden policy to requester-submitted authorization evidence through TPS-side proof generation and on-chain proof verification. The provider registers only a policy commitment on-chain and sends policy-bound proving material to the TPS off-chain. For each access request, the TPS constructs the private witness from the hidden policy material and requester evidence, generates a zero-knowledge proof of policy compliance, and submits a fixed-format proof statement for ACM verification. The ACM records the authorization outcome in contract storage and emits it in the blockchain event log (as access-granted or access-denied events); the data access layer consumes this recorded decision to serve or refuse the request.

This design separates confidentiality-sensitive proof generation from public verification. Policy material and requester evidence remain off-chain—policy material within the provider–TPS boundary, requester evidence between the requester and the TPS—while the blockchain verifies only a fixed-format proof statement against the provider's policy commitment, alongside session parameters validated by the ACM. In a traditional requester-as-prover design, the requester generates the proof itself and must therefore receive the encoded policy and the commitment-opening randomness, so every requester learns the policy in normal operation. In TPS-ZKPI, this material never leaves the provider–TPS boundary: the requester receives no policy material, and policy confidentiality can fail only if the TPS trust assumption itself is violated.

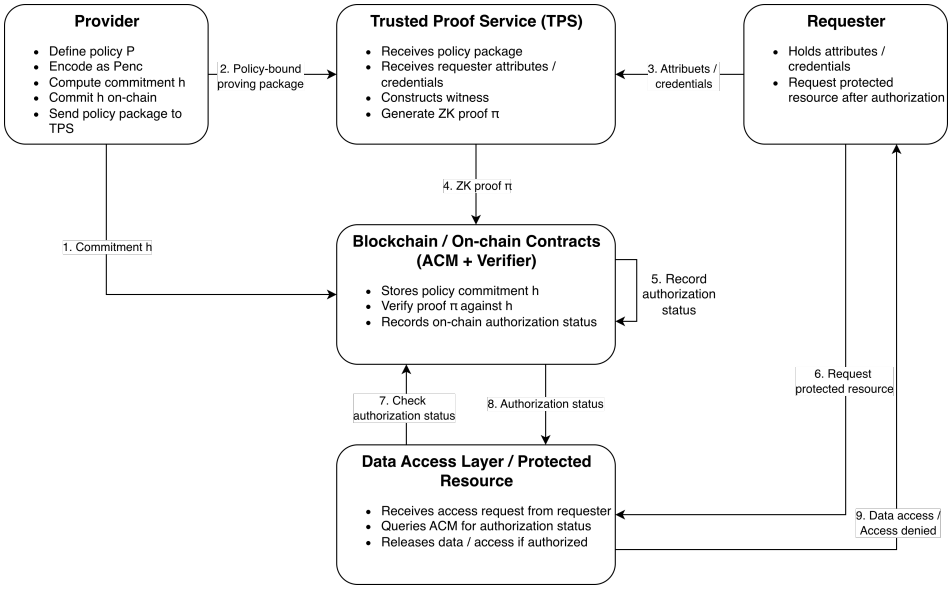



**Figure 1.** TPS-assisted policy-compliance workflow. The provider commits to the encoded policy on-chain and provisions proving material to the TPS (steps 1–2); the TPS constructs the witness from requester evidence and generates the proof (3–4); the ACM verifies the proof and records the outcome (5); the data access layer consults the recorded status to serve the requester (6–9).

### *5.2. Policy Representation and Interpretation*

TPS-ZKPI represents heterogeneous Boolean-based consent policies in a fixed-length, circuit-friendly format. This policy class is chosen for two reasons. First, the targeted biomedical consent conditions—DUO-style data-use terms, disease-scope constraints, and institutional or ethics requirements—are categorical conditions combined with AND/OR/NOT, and conditions that are not natively Boolean, such as temporal validity or credential status, enter as externally attested predicate outcomes. Second, a Boolean policy class admits a fixed-length encoding evaluated by a single reusable interpreter circuit, which keeps the verification interface policy-independent—a prerequisite for the privacy objectives evaluated later. Let

$$X = \{0,1\}^N, \qquad B = \{0,1\}^M \tag{2}$$

denote the requester-attribute assignment space and the external-predicate assignment space, respectively. The full assignment space is

$$O = X \times B, \tag{3}$$

where $a \in X$ encodes requester attributes and $b \in B$ encodes externally attested predicate outcomes, such as credential validity, ontology-linked checks, or obligation-related

conditions. Credential issuance, revocation, and attestation themselves remain external functions (Section 4.1): TPS-ZKPI consumes their outcomes as predicate bits and, where applicable, checks the associated auxiliary witnesses inside the proof relation.

Ontology-linked conditions are represented extensionally rather than through in-circuit ontology reasoning. At encoding time, the provider expands an ontology concept into a whitelist of codes and commits the whitelist as a Merkle root among the policy's per-predicate parameters; in-circuit, the predicate check is a set-membership proof of the requested term against this root. A disease-specific policy scoped to diabetes, for example, whitelists the ICD-10 diabetes subtree, whereas one scoped to type-1 diabetes whitelists only the type-1 codes. Resolving subsumption at encoding time keeps ontology reasoning out of the circuit and preserves the policy-independent interface.

Consent policies are expressed as Boolean formulas over assignment bits:

$$P ::= atom \mid (P \wedge P) \mid (P \vee P) \mid \neg P, \qquad atom ::= x_i \mid y_j. \tag{4}$$

Here $x_i$ selects requester-attribute bit $a_i$ and $y_j$ selects external-predicate bit $b_j$. For example, the consent condition "the resource may be used for disease-specific research on diabetes, or for clinical care under approved clinical terms" is expressed as $(x_1 \wedge y_1) \vee (x_2 \wedge y_2)$, where $x_1$ and $x_2$ flag the disease-specific-research and clinical-care purposes among the requester attributes, $y_1$ attests that the requested disease code lies in the committed diabetes whitelist, and $y_2$ attests that the requested clinical term lies in the approved clinical whitelist. Each policy is compiled into a Reverse Polish Notation (RPN) program over operand tokens and Boolean operator tokens. RPN linearizes arbitrarily nested formulas into a parenthesis-free token sequence that a stack machine evaluates in a single left-to-right pass with a fixed step count—a control structure that maps directly onto a fixed-size circuit and lets one interpreter serve all policies. To support reusable verification, each program is padded to a fixed token length $L$:

$$P_{\text{enc}} = [t_1, \ldots, t_L], \tag{5}$$

together with an explicit program length $\ell \leq L$ carried as a private witness. The interpreter evaluates only the first $\ell$ tokens; padding positions beyond $\ell$ are inactive and leave the output unchanged. The example policy above compiles to the token sequence $[x_1, y_1, \wedge, x_2, y_2, \wedge, \vee]$ with $\ell = 7$; the remaining $L - 7$ positions are padding. The implemented circuit uses $N = 6$, $M = 5$, and $L = \mathsf{DEPTH} = 20$. We denote the set of length-$L$ programs by $\mathsf{Prog}_L$.

The reusable policy interpreter is a zero-knowledge circuit

$$C : \mathsf{Prog}_L \times \{0, \ldots, L\} \times O \to \{0, 1\}. \tag{6}$$

Given an encoded policy, its program length, and operand values, the interpreter evaluates the RPN program using deterministic stack semantics and enforces well-formedness constraints inside the circuit. The circuit outputs 1 only if the requester evidence satisfies the encoded policy. Well-formedness is enforced as hard constraints: for a program that violates the stack discipline, no satisfying witness exists and no proof can be generated. An output of 0 therefore indicates unsatisfied evidence for a well-formed program. The proved acceptance relation is the asserted case $C(P_{\text{enc}}, \ell, (a, b)) = 1$.

The fixed-length interpreter thus realizes the policy-independent interface motivated above, and supports policy evolution through different fixed-length policy programs.

### 5.3. Commitment Binding and Proof Generation

To bind each proof to a provider-defined hidden policy, the provider samples commitment randomness $r$ uniformly from the scalar field and computes

$$h = \mathsf{Com}(P_{\text{enc}}; r) = \mathsf{Poseidon}(P_{\text{enc}} \parallel r), \tag{7}$$

where $h$ is stored on-chain as the public handle of the hidden policy. Under the collision resistance of Poseidon, the commitment binds future proofs to the committed token array $P_{\text{enc}}$. The program length $\ell$ is not part of the commitment preimage: supplying the length recorded at encoding time is part of the prescribed TPS workflow (Section 4.1), and a TPS that deviates from the workflow could evaluate a stack-consistent prefix of the committed array. Binding $\ell$ into the commitment is a straightforward hardening that we leave to future work. Hiding is an explicit commitment assumption (Section 4.1) that relies on high-entropy $r$ and the fixed-length encoding of $P_{\text{enc}}$, which keeps the sponge input unambiguous.

For an access request, the TPS constructs the private witness

$$w = (P_{\text{enc}}, \ell, a, b, r, \omega), \tag{8}$$

where $P_{\text{enc}}$ is the encoded policy, $\ell$ is its program length, $a$ denotes requester attributes, $b$ denotes external predicate outcomes, $r$ is the commitment randomness, and $\omega$ denotes auxiliary witnesses for credential- or predicate-dependent checks. The public statement contains the asserted authorization output, the policy commitment $h$, and a fixed-format parameter commitment $pp$, which binds per-predicate parameters without exposing them in calldata. The requester and resource identifiers and a session nonce are carried as transaction-level parameters that the ACM validates.

For the compiled circuit, the Groth16 [18] setup produces the common reference string as a proving key and a verification key, once for the shared circuit:

$$(pk, vk) \leftarrow \mathsf{Setup}(C). \tag{9}$$

The TPS generates a proof

$$\pi \leftarrow \mathsf{Prove}\big(pk, w; (1, h, pp)\big). \tag{10}$$

The proof certifies the conjunction that $C(P_{\text{enc}}, \ell, (a, b)) = 1$, the hidden policy encoding is consistent with the on-chain commitment $h$, and required external-predicate checks are internally consistent with respect to the supplied parameters and placeholder gadgets. In the current implementation, the issuer-signature check among these gadgets is a hash-based placeholder (a Poseidon consistency check that provides no unforgeability), so attribute-origin binding is not an in-circuit guarantee; its scope and implications are treated in Sections 4 and 7. The proof statement is not bound to the requester, resource, or session context: the ACM validates these as transaction-level parameters, and nonce single-use prevents reuse of the same session parameters only, so replay of a captured proof with fresh parameters by an active submitter is not prevented and lies outside the passive threat model of Section 4. Binding the session context into the public statement is a standard hardening that we leave to future work.

The ACM verifies $\pi$ on-chain against $vk$ and the public statement, and records the outcome of each request: a positive authorization outcome requires both a valid proof and an asserted output of 1; otherwise the request is recorded as a denial. The policy encoding, commitment randomness, requester attributes, credential witnesses, and auxiliary proof material are not posted on-chain.

### *5.4. Transcript Normalization*

Zero-knowledge proofs hide private witness values, but surrounding blockchain artifacts may still leak policy-dependent information. TPS-ZKPI therefore normalizes the public transcript so that different policies induce fixed or tightly bounded observable profiles. Table 3 summarizes the mechanisms.

**Table 3.** Transcript-normalization mechanisms in TPS-ZKPI.

| Artifact | Normalization mechanism |
|---|---|
| Calldata | Fixed verification interface and fixed-size proof/public-input encoding. |
| Logs | Fixed event structure with constant topic and payload layout. |
| Public parameters | Fixed-format handles instead of policy artifacts or credential witnesses. |
| Gas usage | Policy-independent verifier logic and randomized gas padding toward fixed targets. |

These mechanisms do not eliminate all public observability. Rather, they reduce policy-dependent variation in calldata, logs, public parameters, and gas-related behavior that could otherwise serve as a fingerprint for the metadata-only adversary defined in Section 4. TPS-ZKPI does not flatten all residual channels: timing between transactions, transaction ordering, mempool or broadcast patterns, and off-chain TPS communication remain outside the normalized public transcript.

## 6. Privacy Evaluation

We evaluate PCH, PI, and SPP (Section 4.3) against a passive public-chain observer (Section 6.1). The evidence comes from two attack families run on the same executions: a supervised attacker that tries to recover each hidden label from the on-chain observables, and an unsupervised attacker that asks whether the observables cluster along that label without seeing it. Section 6.2 defines both pipelines and the multi-seed criterion that decides each privacy verdict. All three objectives pass, and the unsupervised attacker finds no natural grouping by the hidden labels. Beyond the privacy verdicts, we report the system's performance as a cost profile: per-transaction gas for commit and access (including the padding overhead), calldata sizes, circuit constraint count, and local proving time (Section 6.9, Table 13).

### *6.1. Adversary and observables*

We instantiate the passive public-chain observer of Section 4: an adversary who reads the ledger without restriction and sees every transaction and event our approach emits, but never the plaintext policy or the witness.

Under the assumptions of Section 4.1, the byte values in the adversary's public view carry no policy information: the proof is zero-knowledge, the two commitments in the public statement are hiding, and the authorization output is identical for every execution in the corpus. The attack therefore targets the metadata of these artifacts: for each policy execution (a commit transaction followed by an access transaction) it extracts the following features:

- **Gas:** `commitGasUsed`, `accessGasUsed`.
- **Calldata length:** `commitCalldataLength`, `accessCalldataLength`.
- **Proof size:** `proofSize`.
- **Event logs:** `commitLogsCount`, `commitLogsSize`, `accessLogsCount`, `accessLogsSize`.

Of these features, calldata length, proof size, and log counts/sizes are constant across policies by design; any policy-dependent signal can therefore only flow through the two gas measurements (`commitGasUsed`, `accessGasUsed`). The fixed circuit and the commitment-only interface make the non-gas features constant; randomized gas padding protects the gas channel that remains. The constant features remain in the attack pipeline but carry

no variation to exploit. We do not model side channels such as block timing, transaction ordering, mempool or broadcast patterns, or off-chain TPS communication, nor the non-goals stated in Section 4.

We consider three adversary goals, one per privacy objective: (i) recover the policy's *content* (PCH, Section 6.4); (ii) distinguish two *semantically related* policies (PI, Section 6.5); and (iii) recover the policy's *structure or parameter sizes* (SPP, Section 6.6).

### 6.2. Attack methodology

We evaluate each privacy objective with a primary supervised attack and a complementary unsupervised cross-check. The *supervised* pipeline asks whether a trained classifier can recover a hidden label from the observables, measured by test AUC against the chance level of 0.5; its outcome serves as the main privacy verdict, since the label-informed adversary is at least as strong as the unsupervised one and therefore yields the more conservative test. The *unsupervised* pipeline asks whether the observables cluster along the hidden label, measured by the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) against their nominal chance level of 0, providing a label-free cross-check on the same executions.

**Execution setup.** All policy executions are transactions on the public *Polygon Amoy* testnet: for each generated policy, we commit the encoded policy on-chain from a provider account (`commitPolicy`), generate the Groth16 proof locally in the role of the co-located TPS (Section 4.1), and submit the access transaction from a separate TPS account (`requestAccessBound`). The provider-to-TPS and requester-to-TPS exchanges are in-process calls; they use no public channel and produce no on-chain artifacts, so they do not affect the observables of Section 6.1, which are extracted from the resulting transaction receipts, calldata, and event logs.

Both pipelines run on the same dataset of *Polygon Amoy* executions, collected under the full defense configuration (fixed circuit, commitment-only interface, randomized gas padding). The corpus composition and the two-layer execution design that supplies the balanced PI pairs are detailed in Section 6.3 and Table 4; per-task sample sizes are listed in the result tables. The dataset, attack code, and verdict files are released as an accompanying artifact (anonymized for review).

#### 6.2.1. Supervised attacks

Each objective is operationalized as a set of binary *tasks*, each defined by one hidden label to recover from the observables: detecting the presence of a specific data-use term or disease chapter (PCH, Section 6.4), distinguishing the executions of two policies (PI, Section 6.5), or separating two structural buckets (SPP, Section 6.6). For every task we train a linear and a non-linear classifier (logistic regression, LR; random forest, RF) and report the score that enters the verdict for that objective (defined below). Each task is evaluated over **30 random seeds** using a **policy-disjoint train/test split** (a uniform 70/30 ratio across PCH, PI, and SPP): the policies used to train the attacker never appear in the test set, so a high test AUC reflects leakage that generalizes across policies. For each seed, the split is chosen from repeated random assignments of whole policies to minimize the deviation of each class's test-set proportion from the target ratio (approximate stratification): the policy-disjoint constraint means individual executions cannot be reassigned across the split, so exact stratification is not attainable, but this candidate search keeps per-class test proportions close to 70/30. Here a class is one side of the binary task (the positive or negative value of its hidden label); stratification keeps both classes represented in the test set at roughly their population proportions, so every seed's AUC is estimated on a test set containing both classes. For each seed we run a label-permutation test and record whether

the AUC is significant at $\alpha$=0.05. The classifier is trained once and its test-set probability outputs are fixed; the test-set labels are then shuffled 500 times, and the AUC is recomputed against the fixed probabilities on each shuffle, giving the null distribution of the AUC under no association between scores and labels. The empirical $p$-value is the fraction of shuffles whose AUC meets or exceeds the observed (unshuffled) AUC; the seed is counted as significant when this $p$-value falls below $\alpha$.

For each task, the result tables report the AUC averaged over seeds ($\text{AUC}_{\text{mean}}$) and the mean per-seed distance from chance, $\text{mean}_s\,|\text{AUC}_s - 0.5|$ (reported as the AUC gap). Because the gap averages per-seed absolute deviations, it is generally larger than $|\text{AUC}_{\text{mean}} - 0.5|$ computed from the mean AUC. This AUC gap is an empirical, sample-estimated diagnostic, not the advantage of a formal cryptographic distinguisher and not a proven security bound. Unlike accuracy, whose chance level shifts with the number of classes, AUC is insensitive to class count and prevalence: a one-vs-rest AUC is a prevalence-weighted average of pairwise AUCs, so the results are not tied to the class counts of our generated corpus. Under any number of circulating classes, the one-vs-rest AUC decomposes into pairwise contrasts of the kind that PI and SPP measure directly; if every pairwise contrast sits at chance, so does any prevalence-weighted average of them. The PASS/FAIL decision, defined below, combines $\text{AUC}_{\text{mean}}$ with the significant-seed fraction—the fraction of seeds whose per-seed permutation test rejects at $\alpha$=0.05, reported as %sig in the result tables. The verdict is:

> **FAIL** iff $\text{AUC}_{\text{mean}} > 0.55$ **and** the significant-seed fraction is $\geq 0.5$ (over 30 policy-disjoint seeds); **PASS** otherwise.

Both conditions of this *double criterion* are required: a high AUC that does not reproduce across seeds is not a leak, and neither is a reproducibly significant but near-chance AUC.

Tasks whose test split contains fewer than 20 executions (median over the 30 seeded splits) are *underpowered*—reported but excluded from the verdict, so a small task's sampling noise cannot decide it. The double criterion is applied to LR and RF independently for each task; a task fails if either classifier meets both conditions, and the result tables report the stronger (higher-AUC) classifier for each task.

As established in Section 6.1, the transcript normalization leaves gas as the only policy-dependent channel; the LR/RF feature set is therefore the two gas measurements. A near-chance result alone cannot show whether the defense works or the attacker is simply too weak to find any signal. We therefore pair the main results with the gas-normalization positive control in Section 6.9: the same features, splits, and classifiers recover signal on the un-padded gas transcript, and that signal collapses after padding.

#### 6.2.2. Unsupervised clustering attacks

The clustering attacker drops the labels entirely and asks whether the gas features *naturally* group along the hidden property. We run K-Means (with $K$ set to the number of classes, or $K$=2 for policy pairs) on the on-chain gas features, and score the recovered clusters against the true labels with ARI and NMI. Values near 0 mean the clusters carry no information about the hidden label; values near 1 mean full leakage.

PCH clusters all executions globally (no per-layer splitting), so the large-sample ARI/NMI are read directly. SPP is clustered the same way, once per annotation axis of Section 6.6 with $K$ equal to that axis's bucket count, and its large-sample ARI/NMI are likewise read directly.

For PI, clustering is scored per pair, so each side of a pair must contribute enough executions; this requires two safeguards. **Balanced pairs.** The clustering uses a dedicated pair set built from the same executions: the policies with $\geq 30$ executions each (the depth layer of Section 6.3) are paired into 133 balanced 30-vs-30 contrasts. The pair set of the

supervised PI attack (Section 6.5) is unsuitable for per-pair clustering: most of its pairs hold a single execution per side, and its most imbalanced hold 1 execution against 30, so K-Means cannot recover a meaningful partition, and ARI/NMI $\approx 0$ would reflect the degenerate setup rather than privacy. (The supervised attack itself is unaffected: it pools executions within a pair type rather than partitioning each pair.) **Permutation-debiased NMI.** Even after balancing, each side of a pair holds only 30 executions. At that scale the null simulation in Section 6.8 shows that finite-sample effects alone can raise NMI above zero while ARI stays at chance, so ARI is read directly and NMI must be debiased. The cluster assignments are computed once (K-Means does not see the labels) and the observed NMI is averaged over 20 K-Means seeds; the pair labels are then shuffled 200 times and the NMI recomputed against the fixed assignments on each shuffle, giving the null distribution of NMI under zero true signal. The debiased NMI is the observed NMI minus the mean of this null distribution, and the empirical $p$-value $p_{\mathrm{NMI}}$ is the fraction of shuffles whose NMI meets or exceeds the observed value. The cross-check flags a pair only if its debiased NMI is positive *and* significant ($p_{\mathrm{NMI}} < 0.05$).

### 6.3. Dataset and corpus generation

**Shared corpus.** We evaluate all three objectives on one policy corpus, so the supervised attacks and the clustering cross-check see the same transactions. The corpus holds **1077 distinct policies** run as **2005 on-chain transactions** on *Polygon Amoy*. The policies are produced by a deterministic generator with a fixed seed (`unified-optimized-2025`). For each policy the generator picks its content—a set of DUO data-use terms and, for policies that carry a disease code, one ICD-10 chapter—and its structure: the logical form, nesting depth, predicate count, and whitelist size. For example, one generated policy permits disease-specific and research-specific use (DUO terms DS and RS) and grants access only when *both* of two conditions hold, joined by `AND`: the requested record falls in the ICD-10 *infectious and parasitic diseases* chapter, and the study is a randomized controlled trial—a study-design term from the Ontology for Biomedical Investigations (OBI) [30]. Each condition is one predicate—hence the two predicates—and each is checked against a whitelist of the codes it admits: 12 ICD-10 codes drawn from that chapter and 12 OBI study-type codes, the "medium" parameter-size bucket of Table 4. Table 4 gives the resulting counts for every label and structural bucket.

**Two-layer execution design.** We run the policies in two layers that serve different needs. The *coverage layer* runs 1045 policies once each. It provides breadth, spreading distinct policies across the content labels and structural buckets, so the PCH and SPP attacks have enough different policies to train on one set and test on another. The *depth layer* runs 32 policies 30 times each (960 executions). It provides depth, which the PI attacks need because they compare two policies execution-by-execution, and for that each policy must have multiple executions of its own. This layer is what lets PI form its 133 balanced 30-vs-30 pairs and run the per-pair clustering check (Section 6.2).

**Label restriction and statistical power.** We restrict the generated content labels to those with enough support for a statistically powered attack: six DUO data-use terms and the ten ICD-10 chapters, sampled so that the smallest class reaches at least 30 executions. The complete DUO has 23 data-use terms and ICD-10 has tens of thousands of leaf codes; generating and executing enough policies for every leaf term to reach adequate power would make the on-chain experiment infeasible. Category-level labels are also the right granularity for the privacy question: the observer wants to know whether the *category* of permitted use or disease leaks. This choice is conservative. A coarse label has more executions behind it than a rare leaf term, and more data makes the attack *stronger*; so if the attack cannot recover a coarse label, it would fail even more clearly on a fine one.

Passing at the category level therefore does not hide a leak that finer labels would expose. The restriction also cannot flatter the defenses, which behave the same whatever content a policy carries: the commitment-only interface and gas padding are content-agnostic by construction (Section 5).

**Collection and cleaning.** Executions were collected as live transactions on *Polygon Amoy* under the defense configuration of Section 6.2 (fixed circuit, commitment-only interface, randomized gas padding); observables are extracted from the resulting receipts, calldata, and event logs. One execution is dropped at feature extraction as a cold-start outlier: its gas reflects the one-time initialization and state-creation cost of the first contract call (e.g. writing a storage slot from zero), which later calls avoid by reusing the on-chain state already established. The feature file therefore holds 2004 of the 2005 executions. As established in Section 6.1, transcript normalization leaves gas as the only policy-dependent channel, so the residual per-transaction variation the attacker sees is confined to the two gas measurements.

**Table 4.** Composition of the shared policy corpus (generator seed `unified-optimized-2025`). Policies and executions are counted per label or per structural bucket. DUO terms are *multi-label*: an execution carries every term its policy references, so the per-term counts overlap and sum above the corpus totals; ICD-10 rows cover only the ICD-bearing subset of policies. Each SPP axis partitions the full corpus, so its buckets sum to the corpus totals.

| Grouping | Category | Policies | Executions |
|---|---|---|---|
| *DUO data-use term* (multi-label; counts = policies/executions containing the term) | | | |
| | GR | 197 | 342 |
| | DS | 514 | 920 |
| | RS | 512 | 1121 |
| | POA | 438 | 757 |
| | GS | 417 | 910 |
| | CC | 484 | 1035 |
| *ICD-10 chapter* (ICD-bearing policies only) | | | |
| | ch0 | 50 | 50 |
| | ch1 | 69 | 127 |
| | ch2 | 59 | 117 |
| | ch3 | 54 | 112 |
| | ch4 | 47 | 105 |
| | ch5 | 34 | 92 |
| | ch6 | 49 | 49 |
| | ch7 | 58 | 87 |
| | ch8 | 55 | 113 |
| | ch9 | 39 | 68 |
| | *ICD-bearing subtotal* | 514 | 920 |
| *SPP structureType* | | | |
| | and_only | 246 | 478 |
| | or_only | 347 | 579 |
| | negation_heavy | 292 | 524 |
| | and_or_mixed | 192 | 424 |
| *SPP nestingDepth* | | | |
| | flat | 147 | 263 |
| | shallow | 572 | 1007 |
| | medium | 188 | 304 |
| | deep | 170 | 431 |
| *SPP predicateCombo* | | | |
| | none | 7 | 36 |
| | single | 228 | 344 |
| | few | 672 | 1194 |
| | many | 170 | 431 |
| *SPP paramType* | | | |
| | none | 7 | 36 |
| | small | 200 | 345 |
| | medium | 638 | 1102 |
| | large | 232 | 522 |
| **Corpus total** | | **1077** | **2005** |

**Table 5.** Results on the Polygon Amoy dataset. The supervised attacker uses 30 policy-disjoint seeds and a permutation test. A task passes unless both $AUC_{mean} > 0.55$ and at least 50% of seeds are significant.

| Property | Hidden target | Headline supervised result |
|---|---|---|
| PCH | DUO terms and ICD-10 chapters | 16 content-recovery tasks pass; AUC 0.433–0.561; significant seeds median 2%, max 37% |
| PI | Related policy-pair identity | 4 gating tasks pass; AUC 0.500–0.559; significant seeds median 7%, max 30% |
| SPP | Structure, predicate, parameter, and nesting-depth buckets | 28 pairwise contrasts pass; AUC 0.483–0.629; significant seeds median 3%, max 23% |

### 6.4. Policy Content Hiding (PCH)

**Property.** PCH requires that the public-chain observer cannot recover the *content* of a policy—its DUO data-use terms or the ICD-10 disease codes it references—better than chance.

**Scope of the labels.** The attack targets six DUO data-use terms—general research use (GR), disease-specific research (DS), research-specific restrictions (RS), population origins or ancestry research (POA), geographical restriction (GS), and clinical care use (CC)—and disease codes aggregated to the ten ICD-10 chapters. This label restriction, its statistical-power rationale, and why category-level granularity is a conservative choice are detailed in Section 6.3; the per-label support is given in Table 4.

**Supervised attack.** Each of the six DUO terms (GR, DS, RS, POA, GS, CC) defines a one-vs-rest binary recovery task, on which we run the multi-seed pipeline of Section 6.2. Table 6 reports the per-term verdicts: every DUO task stays below the 0.55 AUC line, with $\mathrm{AUC}_{\mathrm{mean}} \in [0.493, 0.526]$; significant-seed fractions run up to 37% (RS), still short of the 50% gate, and with the AUC leg unmet none approaches the failure criterion. The same pipeline applied to the ten ICD-10 chapters (one-vs-rest over the ICD-bearing rows only, median test $n \approx 270$ per chapter; Table 7) passes all ten. Two chapters edge above the AUC line—ch6 ($\mathrm{AUC}_{\mathrm{mean}} = 0.561 \pm 0.053$, 7% significant seeds) and ch7 ($0.552 \pm 0.047$, 20%)—but neither meets the reproducibility condition (%sig $< 50\%$), so both are boundary cases rather than stable channels. We report such cases rather than suppress them.

**Table 6.** Policy Content Hiding: per-term supervised attack over 30 policy-disjoint seeds. Each DUO term is a one-vs-rest binary task. AUC is the higher-scoring of logistic regression and random forest per task (Section 6.2). *%sig* is the fraction of seeds whose permutation test rejects at $\alpha$=0.05. All six tasks pass the unified criterion.

| Term | AUC (mean ± std) | AUC gap (mean) | %sig | med $n$ |
|---|---|---|---|---|
| DUO GR | $0.507 \pm 0.027$ | 0.024 | 3% | 613 |
| DUO DS | $0.493 \pm 0.018$ | 0.015 | 0% | 613 |
| DUO RS | $0.526 \pm 0.022$ | 0.029 | 37% | 613 |
| DUO POA | $0.496 \pm 0.018$ | 0.015 | 3% | 613 |
| DUO GS | $0.523 \pm 0.020$ | 0.027 | 23% | 613 |
| DUO CC | $0.495 \pm 0.026$ | 0.019 | 0% | 613 |

**Table 7.** Policy Content Hiding: per-chapter ICD-10 supervised attack over 30 policy-disjoint seeds (one-vs-rest, ICD-bearing rows only; median test $n \approx 270$). AUC is the higher-scoring of logistic regression and random forest per chapter (Section 6.2). Two chapters exceed the 0.55 AUC line—ch6 (AUC 0.561, %sig 7%) and ch7 (AUC 0.552, %sig 20%)—but neither meets the 50% significant-seed reproducibility condition, so all ten chapters pass.

| Chapter | AUC (mean ± std) | AUC gap (mean) | %sig | med $n$ |
|---|---|---|---|---|
| ICD ch0 | $0.433 \pm 0.060$ | 0.075 | 0% | 270 |
| ICD ch1 | $0.455 \pm 0.066$ | 0.064 | 3% | 270 |
| ICD ch2 | $0.507 \pm 0.029$ | 0.024 | 0% | 270 |
| ICD ch3 | $0.486 \pm 0.033$ | 0.031 | 0% | 284.5 |
| ICD ch4 | $0.541 \pm 0.039$ | 0.047 | 13% | 270 |
| ICD ch5 | $0.503 \pm 0.031$ | 0.025 | 0% | 270 |
| ICD ch6 | $0.561 \pm 0.053$ | 0.069 | 7% | 270 |
| ICD ch7 | $0.552 \pm 0.047$ | 0.058 | 20% | 270 |
| ICD ch8 | $0.516 \pm 0.021$ | 0.021 | 0% | 270 |
| ICD ch9 | $0.492 \pm 0.040$ | 0.034 | 0% | 270 |

### 6.5. Policy Indistinguishability (PI)

**Property.** PI strengthens PCH: even when two policies are *semantically related*, the observer must not tell which one was executed—formally, $\Pr[\hat{b} = b] \approx \frac{1}{2}$. We construct policy pairs of four types, defined by semantic similarity and by whether structure or execution protections differ: `semantic_equivalent` (near-identical semantics, similarity $\geq 0.999$, but different structure), `structural_variant` (the same, plus differing execution protections), `weak_equivalent` ($\geq 0.95$ semantic overlap), and `controlled_difference` (similarity $< 0.95$, the least similar pairs by construction; reported for reference but excluded from the verdict). Semantic similarity here is a fixed heuristic score over policy semantics—the Jaccard overlap [31] of DUO terms combined with agreement of the ICD-10, OBI, and other scope fields—that returns 1 when two policies share an identical normalized logic string. The $\geq 0.999$ threshold therefore selects logically identical policies and $\geq 0.95$ their close approximations; the exact component weights are given in the released artifact. To build the pairs we score every candidate policy couple by this similarity, assign it to one of the four types above, and sample a fixed budget of pairs per type; each retained pair is an ordered couple $(P_0, P_1)$ that the supervised attack below tries to tell apart.

**Supervised attack.** Each pair is an ordered couple $(P_0, P_1)$, and the executed policy defines a binary label $b \in \{0, 1\}$; the attacker's task is to predict $b$ from the on-chain features of a single execution, i.e. to decide which policy of the pair produced it. Indistinguishability is thus the two-class problem $\Pr[\hat{b} = b] \approx \frac{1}{2}$, corresponding to a classifier AUC of 0.5. We train a logistic-regression and a random-forest classifier and, per pair type, report the higher-AUC one, following the convention of Section 6.2; the random forest leads on every pair type. We evaluate this single-execution attacker over 30 policy-disjoint seeds (Table 8). The overall AUC is $0.514 \pm 0.022$ with only 7% of seeds significant. Among the three pair types that enter the PI verdict, `structural_variant` (0.514) and `weak_equivalent` (0.500) sit at chance, while `semantic_equivalent` is the only one to edge above the 0.55 line, at 0.559. That elevation is driven by a minority of seeds—30% reach permutation significance, below the 50% fraction the verdict requires to declare a leak—so PI still passes. `controlled_difference`, the least similar pairs by construction and hence the easiest to separate, is excluded from the verdict and reported for reference: it carries the largest residual signal (0.562, 20% of seeds significant). Padding does not erase this residual, but the ablation shows it substantially reduces the otherwise-recoverable gas signal relative to unpadded execution (Section 6.9).

**Table 8.** Policy Indistinguishability: single-sample distinguisher (Level I) over 30 policy-disjoint seeds, by policy-pair type. AUC is the higher-scoring of logistic regression and random forest per pair type (Section 6.2). `controlled_difference` is reported for reference and excluded from the PI verdict.

| Pair type | AUC (mean ± std) | AUC gap (mean) | %sig | med $n$ |
|---|---|---|---|---|
| overall | $0.514 \pm 0.022$ | 0.021 | 7% | 415.5 |
| semantic_equivalent | $0.559 \pm 0.063$ | 0.069 | 30% | 118 |
| structural_variant | $0.514 \pm 0.061$ | 0.050 | 7% | 68 |
| weak_equivalent | $0.500 \pm 0.057$ | 0.044 | 3% | 92 |
| controlled_difference [†] | $0.562 \pm 0.037$ | 0.066 | 20% | 118 |

[†] Reported for reference; excluded from the PI verdict.

### 6.6. Structure & Parameter Privacy (SPP)

**Property.** SPP requires that neither the *shape* of a policy (logical structure, nesting depth, predicate coverage) nor the *size* of its parameters (whitelist/set sizes) leaks to the observer. The supervised attack organizes the test along four annotation axes—logical structure type (`structureType`), predicate count (`predicateCombo`), parameter size (`paramType`), and nesting depth (`nestingDepth`)—and attacks each pair of buckets within an axis.

`structureType` has five buckets: `trivial` (a single predicate with no boolean connective, e.g. policy logic `"GR"`), `and_only`, `or_only`, `and_or_mixed`, and `negation_heavy`. `predicateCombo` buckets policies by predicate-namespace count into `none` (0), `single` (1), `few` (2–3), and `many` ($\geq$ 4); `paramType` buckets by whitelist size into `none` (no whitelist), `small` (1–4), `medium` (5–16), and `large` ($\geq$ 17); `nestingDepth` buckets by structural nesting into `flat`, `shallow`, `medium`, and `deep`. The per-bucket policy and execution counts are listed in Table 4. Contrasting every within-axis bucket pair gives $10 + 6 + 6 + 6 = 28$ contrasts.

**Supervised attack.** Table 9 reports all 28 pairwise contrasts over 30 policy-disjoint seeds, with the per-contrast AUC taken from the higher-scoring of the two classifiers following the convention of Section 6.2. Most contrasts sit at chance, with $\text{AUC}_{\text{mean}} \in [0.48, 0.54]$. Four exceed the 0.55 line—all separating the tiny `none`-predicate or `none`-parameter class from a populated one: `none/few` (0.629), `none/many` (0.617), `paramType none/medium` (0.612), and `none/single` (0.582). These buckets hold very few minority-class executions, so an individual split can look separable; but across the 30 seeds their significant-seed fractions stay at or below 20%—and no contrast exceeds 23% overall—far short of the 50% the criterion requires. All 28 therefore pass and SPP passes overall. Reporting AUC rather than an accuracy-style score already discounts majority-class prediction (Section 6.2); the residual elevation on the `none` contrasts reflects their small, low-power minority class rather than a stable structural or parameter-size channel. Section 6.9 provides the corresponding positive control: when gas padding is removed, the same SPP attacker recovers structural and parameter-size signal, and that signal collapses under the full normalization.

**Table 9.** Structure & Parameter Privacy: 28 pairwise contrasts across four annotation axes, each over 30 policy-disjoint seeds. AUC is the higher-scoring of logistic regression and random forest per contrast (the stronger-attacker classifier used for the verdict). Every contrast passes: the four contrasts whose AUC exceeds 0.55—all separating the tiny `none`-predicate or `none`-parameter class from a populated one—reach at most 20% significant seeds, far short of the 50% gate.

| Contrast | AUC (mean ± std) | AUC gap (mean) | %sig | med $n$ |
|---|---|---|---|---|
| *structureType* | | | | |
| trivial / or_only | 0.539 ± 0.043 | 0.042 | 23% | 156 |
| trivial / and_only | 0.535 ± 0.044 | 0.042 | 3% | 78 |
| trivial / negation_heavy | 0.508 ± 0.046 | 0.034 | 3% | 138 |
| trivial / and_or_mixed | 0.533 ± 0.047 | 0.047 | 13% | 108 |
| and_only / or_only | 0.531 ± 0.050 | 0.045 | 10% | 140 |
| or_only / negation_heavy | 0.520 ± 0.036 | 0.033 | 10% | 202 |
| or_only / and_or_mixed | 0.489 ± 0.026 | 0.024 | 0% | 170 |
| and_only / negation_heavy | 0.512 ± 0.041 | 0.035 | 3% | 125 |
| and_only / and_or_mixed | 0.513 ± 0.059 | 0.051 | 0% | 95 |
| negation_heavy / and_or_mixed | 0.516 ± 0.044 | 0.038 | 7% | 155 |
| *predicateCombo* | | | | |
| none / single | 0.582 ± 0.129 | 0.130 | 0% | 74 |
| none / few | 0.629 ± 0.099 | 0.138 | 20% | 216 |
| none / many | 0.617 ± 0.106 | 0.119 | 13% | 59 |
| single / few | 0.515 ± 0.036 | 0.031 | 7% | 284 |
| single / many | 0.500 ± 0.039 | 0.029 | 0% | 127 |
| few / many | 0.483 ± 0.029 | 0.026 | 0% | 269 |
| *paramType* | | | | |
| none / small | 0.543 ± 0.115 | 0.096 | 3% | 66 |
| none / medium | 0.612 ± 0.080 | 0.125 | 17% | 204 |
| none / large | 0.545 ± 0.093 | 0.086 | 0% | 78 |
| small / medium | 0.521 ± 0.033 | 0.030 | 10% | 263 |
| small / large | 0.513 ± 0.048 | 0.036 | 0% | 138 |
| medium / large | 0.486 ± 0.023 | 0.022 | 0% | 277 |
| *nestingDepth* | | | | |
| flat / shallow | 0.522 ± 0.040 | 0.037 | 13% | 228 |
| flat / medium | 0.499 ± 0.043 | 0.036 | 3% | 106 |
| flat / deep | 0.490 ± 0.033 | 0.030 | 0% | 103 |
| shallow / medium | 0.511 ± 0.028 | 0.023 | 0% | 240 |
| shallow / deep | 0.492 ± 0.032 | 0.026 | 0% | 237 |
| medium / deep | 0.492 ± 0.040 | 0.032 | 0% | 115 |

### *6.7. Unsupervised clustering: a label-free cross-check*

The supervised attacks (Sections 6.4–6.6) assume an adversary who can train on labeled executions. A weaker but assumption-free attacker simply asks whether the on-chain gas features *cluster* along the hidden property with no supervision at all. Table 10 consolidates this attack across all three objectives: K-Means recovers essentially nothing, with ARI within ±0.002 and NMI below 0.02 for every DUO, PI, and SPP target. NMI values are interpreted against the null finite-sample simulation in Section 6.8, which shows that the slightly higher ICD-10 NMI is consistent with small-sample inflation rather than recoverable signal. The unsupervised attacker agrees with the supervised verdict—privacy holds—as a second, label-free attack family on the same dataset (Figure 2).

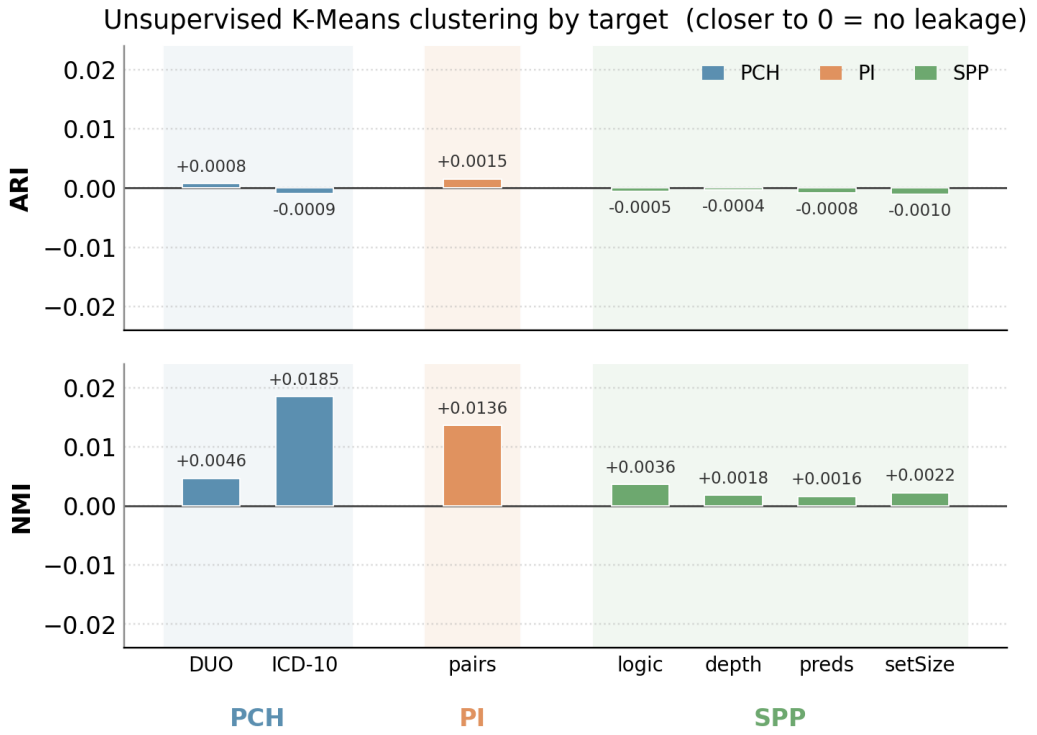



**Figure 2.** Unsupervised K-Means clustering across all three objectives on a common axis: ARI (top) and NMI (bottom) for every PCH, PI, and SPP target. Every bar sits at the zero line—the natural gas clusters carry no information about the hidden label. The SPP bar labels use the raw clustering fields, which map to the supervised axes of Section 6.6 as `logicStructure`=`structureType`, `predicateCount`=`predicateCombo`, and `setSize`=`paramType`; `nestingDepth` is a supervised axis under its own name. NMI is read against the null finite-sample simulation in Section 6.8.

**Removing sample-size confounds.** The PI clustering inputs are rebalanced into 30-vs-30 contrasts and permutation-debiased as described in Section 6.2, so the near-zero ARI/NMI is not a degenerate-measurement artifact. The conclusion matches the supervised PI verdict (Section 6.5), and the ablation (Section 6.9) confirms the clustering device itself is live: significance appears on the un-padded gas transcript and disappears after normalization.

**Table 10.** Unsupervised clustering attack (K-Means, $K$=number of classes / $K$=2 for pairs) on the on-chain gas features. ARI and NMI near 0 mean the natural gas clusters carry no information about the hidden label; NMI is interpreted against the null simulation in Section 6.8. PCH clusters all executions globally (no layering).

| Property | Target | $n$ | #cls. | ARI | NMI |
|---|---|---|---|---|---|
| PCH | DUO terms | 2005 | 6 | 0.0008 | 0.0046 |
| PCH | ICD-10 chapters | 920 | 10 | −0.0009 | 0.0185 |
| PI | policy pairs (30v30) | 133 | 2 | 0.0015 | 0.0136 |
| SPP | `structureType` | 1141 | 5 | −0.0005 | 0.0036 |
| SPP | `predicateCombo` | 1141 | 4 | −0.0008 | 0.0016 |
| SPP | `paramType` | 1141 | 4 | −0.0010 | 0.0022 |
| SPP | `nestingDepth` | 1141 | 4 | −0.0004 | 0.0018 |

### *6.8. Sample-size sensitivity of NMI*

The clustering cross-check (Section 6.7) treats ARI as the primary conservative indicator because NMI can take moderately high values under small samples. To justify that choice, we quantify how *finite-sample effects* alone can inflate NMI even when there is *no* discriminative signal, using an idealized null in which all classes are statistically indistinguishable.

**Null hypothesis.** We sample every class from the *same* distribution, $x_i \sim \mathcal{N}(0, I_d)$ for all classes, where $d$ is the synthetic feature dimension and $I_d$ is the $d$-dimensional identity covariance. Since no class carries class-specific information, a clustering algorithm should not recover class membership, and both NMI and ARI should concentrate around 0 as the sample size grows.

**Simulation.** For each configuration we run $T = 1000$ independent trials. In each trial we (i) generate $C$ classes with $n$ i.i.d. samples per class from $\mathcal{N}(0, I_d)$, assigning each sample a class label by construction order (these labels are independent of the sample values, since all classes share one distribution), (ii) apply K-Means with $K = C$ to the pooled data, and (iii) score the predicted assignments against these assigned class labels with NMI and ARI. We sweep $n$ while holding $C$ fixed, and report the trial means $\overline{\text{NMI}}$ and $\overline{\text{ARI}}$. We consider a **PI-like** binary structure ($C = 2$, $n \in \{3, 10, 50, 100\}$) and an **ICD10-like** multi-class structure ($C = 5$, $n \in \{3, 10, 100\}$).

**Results.** Table 11 shows that, although all classes are indistinguishable by construction, **NMI is noticeably positive at very small $n$ but collapses toward zero as $n$ grows**, whereas **ARI stays near zero throughout**. For the binary structure, $\overline{\text{NMI}}$ falls from 0.1808 at $n$=3 ($N$=6) to 0.0404 at $n$=10, and to 0.0066/0.0038 at $n$=50/100, while $\overline{\text{ARI}}$ stays at $-0.0190$, $-0.0020$, $-0.0011$, 0.0002. For the multi-class structure the same trend holds: $\overline{\text{NMI}}$ goes from 0.4168 ($n$=3) to 0.1249 ($n$=10) to 0.0105 ($n$=100), with $\overline{\text{ARI}}$ near zero (0.0005, 0.0018, $-0.0000$). Hence moderately high NMI in small-sample clustering should not be read as actionable leakage, and ARI is the more conservative indicator of practical recoverability in our unsupervised evaluation.

**Table 11.** Sample-size sensitivity of NMI and ARI under the null hypothesis where all classes are i.i.d. sampled from $\mathcal{N}(0, I_d)$ ($T$=1000 trials per row).

| Structure | $n$ (per class) | $C$ | $N$ | $\overline{\text{NMI}}$ | $\overline{\text{ARI}}$ |
|---|---|---|---|---|---|
| PI-like | 3 | 2 | 6 | 0.1808 | −0.0190 |
| PI-like | 10 | 2 | 20 | 0.0404 | −0.0020 |
| PI-like | 50 | 2 | 100 | 0.0066 | −0.0011 |
| PI-like | 100 | 2 | 200 | 0.0038 | 0.0002 |
| ICD10-like | 3 | 5 | 15 | 0.4168 | 0.0005 |
| ICD10-like | 10 | 5 | 50 | 0.1249 | 0.0018 |
| ICD10-like | 100 | 5 | 500 | 0.0105 | −0.0000 |

### *6.9. Ablation: gas-normalization positive control*

The main results show that the full mechanism hides content, policy identity, and structure/parameters. The ablation asks *why*: how much of the residual attack surface is removed by randomized gas padding after the fixed circuit and commitment-only interface have already eliminated calldata, proof-size, and event-log channels.

We compare two aligned protection profiles on the same selected policy set. **B1** uses the fixed circuit, commitment-only interface, and TPS execution profile without commit/access gas padding. **B2** is the full mechanism used in the main evaluation, with randomized gas padding. This is the positive control for the gas channel: calldata, proof size, and event-log shape are already fixed, so B1 is an un-padded gas-transcript control rather than a run with every normalization layer removed. The same features, splits, and classifiers produce well-above-chance AUCs on B1 when padding is absent, while B2 returns the attacker toward chance. Thus the main privacy claim rests on a real ablation contrast rather than on a synthetic label derived from an input feature. The paired leakage metric is

$$\Delta_{\text{leak}} = |\text{AUC}_{\text{B1}} - 0.5| - |\text{AUC}_{\text{B2}} - 0.5|.$$

Positive values mean padding moved the attacker closer to chance, and we bound each $\Delta_{\text{leak}}$ with a 95% stability interval over 30 paired train/test splits. Table 12 is intentionally separated from the main privacy verdicts: the main verdicts answer whether the full mechanism is private; this ablation attributes that privacy to a specific defense layer and reports its gas cost.

**Padding removes otherwise-recoverable signal.** Figure 3 reports the *full* set of supervised contrasts: each point places a contrast at its unpadded deviation from chance $|\text{AUC}_{\text{B1}} - 0.5|$ against its padded deviation $|\text{AUC}_{\text{B2}} - 0.5|$, so points below the diagonal are contrasts where padding moved the attacker back toward chance. The distribution is asymmetric in exactly the way an effective defense predicts. Every contrast on which B1 carries a sizable signal ($|\text{AUC}_{\text{B1}} - 0.5| > 0.15$) collapses toward the padded axis under B2. The two strongest SPP contrasts illustrate this: `nestingDepth flat/deep` falls from AUC 0.85 to 0.55, and `structureType trivial/negation_heavy` from 0.84 to 0.52, each with a $\Delta_{\text{leak}}$ whose 95% interval excludes zero (a statistically significant reduction). Across the 49 random-forest contrasts the median unpadded deviation of 0.136 drops to 0.033 under padding, and the per-family median $\Delta_{\text{leak}}$ is $+0.169$ (SPP), $+0.072$ (PCH), and $+0.039$ (PI). The clustering attacker shows the same pattern on a second, label-free attack family: PI pairwise clustering reaches debiased significance on 68% of pairs under B1 (mean ARI 0.50) versus 5% under B2 (mean ARI 0.0015), and SPP logic-structure clustering drops from ARI 0.043 (B1) to $-0.001$ (B2). This attributes the gas-channel protection reported in Sections 6.4–6.7 to randomized gas padding, rather than to weak attacks or insufficient sample size.

The effect is conditional, and we report it as such. Padding reduces leakage wherever a recoverable gas-channel signal exists and is otherwise inert: the 13 random-forest contrasts with no detectable effect ($\Delta_{\text{leak}}$ interval spanning zero) sit in the lower-left of Figure 3, where B1 is already near chance so there is little to remove. Padding also removes a substantial PCH gas-channel signal that is recoverable when it is absent: the per-family RF median $\Delta_{\text{leak}}$ is $+0.072$, driven by chapters such as ICD ch8 (AUC 0.92 $\rightarrow$ 0.41) and ch3 (0.79 $\rightarrow$ 0.41) and terms such as DUO GR (0.70 $\rightarrow$ 0.51) and `DUO POA` (0.61 $\rightarrow$ 0.49). This complements, rather than replaces, the commitment-only interface and content-independent transcript normalization on which content hiding rests in the deployed system. No random-forest contrast shows a statistically significant *increase* in leakage (0 of 49); the only two contrasts whose interval falls entirely below zero are logistic-regression fits on the tiny `none`-bucket SPP contrasts (`predicateCombo none/few` and `paramType none/medium`), where LR over-separates on a near-degenerate minority class rather than padding creating leakage, and neither is flagged as a leak by the multi-seed verdict. These per-contrast counts are descriptive, not multiplicity-corrected; the causal claim rests on the direction and magnitude of the shift where a signal is present, corroborated by the independent clustering attacker. The logistic-regression panel (shown for robustness only, since it over-separates on the near-constant execution features) exhibits the same below-diagonal shift.

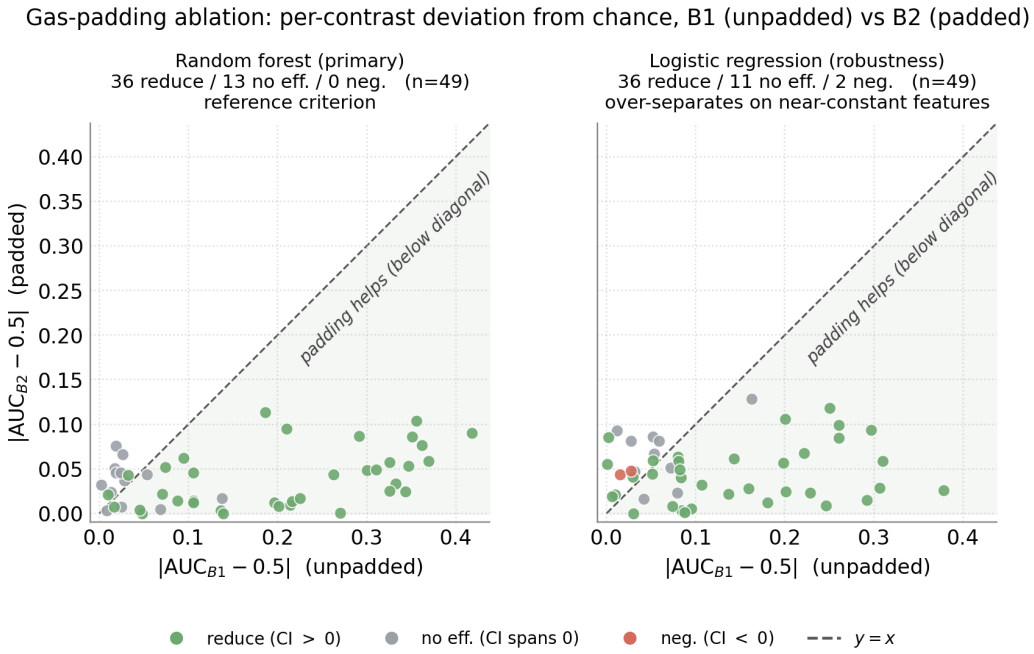



**Figure 3.** Gas-padding ablation over *all* supervised contrasts. Each point is one contrast at its unpadded deviation from chance $|\mathrm{AUC}_{\mathrm{B1}} - 0.5|$ (x) versus its padded deviation $|\mathrm{AUC}_{\mathrm{B2}} - 0.5|$ (y); points below the $y = x$ diagonal are cases where randomized gas padding moved the attacker back toward chance. Color encodes the sign of the bootstrap 95% CI of $\Delta_{\mathrm{leak}}$: reduce (CI$> 0$), no effect (CI spans 0), negative (CI$< 0$). Every contrast with a sizable unpadded signal collapses toward the padded axis, while the few negative cases sit in the lower-left, where B1 is already near chance. The random-forest panel is the reference criterion; logistic regression is shown only for robustness, as it over-separates on the near-constant execution features.

**Cost.** Padding is not free: it raises mean commit gas from 216,562 (B1) to 303,556 (B2), a 40.2% increase, and mean access gas from 546,876 to 753,310, a 37.7% increase. This overhead is the principal efficiency tradeoff of the design. Table 13 adds a single-point DEPTH $= 20$ cost characterization: the circuit compiles to 75,097 R1CS constraints, and mean local proof generation takes 3,881.5 ms, preceded by 190.8 ms of witness generation (about 4.1 s end to end). Gas, logs, and transaction objects are taken from the Polygon Amoy executed datasets; transaction hashes are stored in the B1/B2 executed datasets. Calldata lengths are the on-chain transaction input sizes fetched by hash from Polygon Amoy, and are constant across policies (B1 commit/access 164/452 bytes, B2 196/484). Proving time is local prover timing from the B1 run, not an on-chain measurement. Cost-vs-DEPTH scaling is left to future work because only DEPTH $= 20$ is compiled and evaluated here.

**Table 12.** Gas-padding ablation on Polygon Amoy: representative contrasts illustrating the padding effect. B1 (no gas padding) and B2 (deployed, randomized gas padding) are aligned on the same policy set. Supervised AUC is the random-forest score (the stronger of the two classifiers here; see Section 6.2); clustering reports K-Means ARI. $\Delta_{\text{leak}} > 0$ (defined in the text) means padding moved the attacker toward chance, reported as the mean with a 95% stability interval over 30 paired splits. Each supervised row is, per experiment, the largest-signal contrast whose $\Delta_{\text{leak}}$ interval excludes zero (two for SPP), chosen to avoid cherry-picking. The full per-contrast distribution is in Figure 3.

| Attack | Target | B1 (unpad.) | B2 (pad.) |
|---|---|---|---|
| **Supervised** (RF AUC; $\Delta_{\text{leak}}$ with 95% interval) | | | |
| SPP | `nestingDepth flat/deep` | 0.847 | 0.553 |
| | $\Delta_{\text{leak}} = 0.280$ CI$[0.254, 0.304]$ | | |
| SPP | `structureType trivial/negation_heavy` | 0.843 | 0.524 |
| | $\Delta_{\text{leak}} = 0.276$ CI$[0.248, 0.301]$ | | |
| PCH | ICD ch8 | 0.918 | 0.410 |
| | $\Delta_{\text{leak}} = 0.327$ CI$[0.302, 0.349]$ | | |
| PI | `semantic_equivalent` † | 0.361 | 0.500 |
| | $\Delta_{\text{leak}} = 0.140$ CI$[0.084, 0.205]$ | | |
| **Unsupervised** (K-Means ARI) | | | |
| PCH | ICD-10 chapter | 0.064 | −0.005 |
| SPP | logic structure | 0.043 | −0.001 |
| SPP | predicate count | 0.054 | 0.001 |
| **Gas cost** (mean, B1 → B2) | | | |
| Commit gas | | 216,562 | 303,556 (+40.2%) |
| Access gas | | 546,876 | 753,310 (+37.7%) |

† B1 sits below 0.5: the unpadded attacker separates the pair in the inverted direction (still 0.14 from chance). $\Delta_{\text{leak}}$ is defined on $|\text{AUC} - 0.5|$.

**Table 13.** Single-point cost profile at DEPTH = 20. Gas, logs, and transaction objects are measured from Polygon Amoy executed datasets; proving time is local prover timing from the B1 run. Calldata length is the on-chain transaction input size fetched by hash from Polygon Amoy; it is constant across all policies in each profile.

| Metric | Value / source |
|---|---|
| Circuit size | 75,097 R1CS constraints; 74,834 wires; DEPTH = 20 |
| Public interface | 2 public inputs plus 1 output |
| Verifier gas, unpadded B1 | Commit 216,562; access 546,876 mean gas |
| Verifier gas, padded B2 | Commit 303,556; access 753,310 mean gas |
| Event logs | Commit: 1 log / 180 bytes; access: 2 logs / 296 bytes |
| Calldata length | B1: commit 164, access 452 bytes; B2: commit 196, access 484 bytes (constant across policies) |
| Local witness time | 190.8 ms mean (B1) |
| Local prove time | 3881.5 ms mean (B1) |

## 7. Discussion

The evaluation indicates that TPS-ZKPI reduces the policy-dependent signals in normalized public-chain artifacts below our multi-seed leakage criterion. Across all PCH, PI, and SPP tasks, supervised classifiers stay below that criterion, and unsupervised clustering shows no recoverable structure: ARI stays within ±0.002 of chance on every target, and the one elevated NMI (ICD-10) is consistent with the finite-sample inflation quantified by the null simulation of Section 6.8. The gas-padding ablation (Section 6.9) sharpens

this conclusion: the same attacks become measurably effective once gas padding is removed. This attributes the gas-channel protection specifically to randomized gas padding. Content hiding itself rests on the commitment-only interface and the content-independent transcript normalization, with padding additionally suppressing the residual gas-channel signal that is otherwise recoverable for some content labels. Together with fixed-length policy representation, reusable interpretation, and commitment binding, these mechanisms limit the extent to which public-chain observers infer hidden policy content or structural properties from authorization transactions.

These results support a scoped privacy claim. The evaluation focuses on a metadata-only public-chain adversary that observes commitments, proofs, public parameters, calldata, events, and gas-related artifacts, but not TPS-side witness construction or off-chain policy material. Within this scope, the residual cost is the randomized padding's gas overhead: about 38% on access gas and 40% on commit gas (Section 6.9).

Boundary cases above the 0.55 AUC line do occur—PCH chapters ch6 and ch7, and the PI `semantic_equivalent` pairs—but none reproduces across seeds, and we report them. Stronger adversaries—compromised proof services, adaptive requester-side probing, or off-chain side channels—remain outside this scope and motivate the reduced-trust extensions discussed below.

The TPS trust assumption is the main architectural tradeoff. Moving confidentiality-sensitive proof generation to the TPS keeps policy-bound proving information away from requesters and public smart contracts while preserving on-chain verifiability, but it shifts part of the confidentiality burden to the service layer; the provider-operated TPS used here concentrates that burden at the provider rather than at an independent service (Section 4). This tradeoff leaves on-chain verifiability intact but does affect liveness and accountability: the TPS or provider can refuse to produce a proof, and such a refusal may be indistinguishable from a legitimate denial at the on-chain interface.

The same trust concentration underlies the attribute-origin limitation noted in Section 4. Because the TPS constructs the witness, a compromised TPS, or a TPS colluding with the provider, can substitute attribute values and still produce a verifying proof. Enforcing attribute-origin binding in-circuit would require the requester to reveal attributes in clear to the TPS for issuer-signature verification, which reintroduces the same confidentiality concession. Reconciling attribute-origin binding with attribute confidentiality under a single TPS is therefore a structural limitation of the current design.

Two further gaps concern the soundness of the proof statement, and the privacy results above do not speak to them. First, the program length is not part of the commitment preimage (Section 5), so a TPS that deviates from the prescribed workflow could evaluate a stack-consistent prefix of the committed array instead of the full policy. Second, the proof statement is not bound to the requester, resource, or session context (Section 5); the ACM validates these as transaction-level parameters and enforces nonce single-use, so an active adversary that captures a proof cannot reuse the same session parameters, but could replay it with fresh ones. Both are limitations of the current instantiation, not of the approach: committing to the program length and binding the session context into the public statement are standard hardenings. Neither affects the metadata-only adversary evaluated here—both require an actively deviating TPS or an active on-chain adversary—but both must be closed before deployment.

Three directions follow. First, the two soundness gaps above should be closed in-circuit, by including the program length in the commitment preimage and lifting the session context into the proof statement; the latter changes the public-input layout and therefore the verifier gas, so the cost profile of Table 13 would need to be re-measured. Second, accountable proof obligations—for example, on-chain commitments to serve re-

quests together with timeout-with-evidence mechanisms—would make proof refusals attributable. Third, the reduced-trust TPS instantiations of Section 4 (collaborative-SNARK or TEE-backed proving) would relax the trust concentration itself, including the attribute-origin limitation; realizing and evaluating them is left to future work.

## 8. Conclusion

This paper studied policy-private consent enforcement in blockchain-based healthcare data sharing under public-chain observability and an explicit TPS trust assumption. We proposed TPS-ZKPI, a Trusted Proof Service-assisted zero-knowledge approach that binds hidden provider-defined policies to on-chain commitments, evaluates Boolean-based consent policies through a reusable policy interpreter, and normalizes public verification artifacts to reduce metadata-level leakage. Across supervised and unsupervised attacks on PCH, PI, and SPP, no modeled adversary recovered hidden policy information reproducibly above chance under our multi-seed leakage criterion, and a paired gas-padding ablation attributed the gas-channel protection specifically to randomized gas padding—at a gas overhead of about 38% on access and 40% on commit.

These results hold for a metadata-only public-chain adversary, and the privacy claim is empirical. Two soundness gaps in the current instantiation remain open: the committed policy does not bind the program length, and the proof statement is not bound to the session context (Section 7). Closing both, together with extending the analysis to stronger adaptive adversaries, off-chain leakage, richer policy languages, reduced-trust proof generation, and privacy-preserving resource discovery, is left to future work.

**Author Contributions:** TODO.

**Funding:** TODO.

**Institutional Review Board Statement:** Not applicable.

**Informed Consent Statement:** Not applicable.

**Data Availability Statement:** TODO.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## Abbreviations

The following abbreviations are used in this manuscript:

| | |
|---|---|
| TPS-ZKPI | Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter |
| TPS | Trusted Proof Service |
| ACM | Access-Control Manager |
| ZKP | Zero-Knowledge Proof |
| SNARK | Succinct Non-interactive Argument of Knowledge |
| TEE | Trusted Execution Environment |
| RPN | Reverse Polish Notation |
| PCH | Policy Content Hiding |
| PI | Policy Indistinguishability |
| SPP | Structure and Parameter Privacy |
| DUO | Data Use Ontology |
| ICD-10 | International Classification of Diseases, 10th Revision |
| OBI | Ontology for Biomedical Investigations |
| GA4GH | Global Alliance for Genomics and Health |
| CP-ABE | Ciphertext-Policy Attribute-Based Encryption |
| PRE | Proxy Re-Encryption |
| AUC | Area Under the (ROC) Curve |
| ARI | Adjusted Rand Index |
| NMI | Normalized Mutual Information |
| LR | Logistic Regression |
| RF | Random Forest |

## References


1. Kakarlapudi, P.V.; Mahmoud, Q.H. A systematic review of blockchain for consent management. In Proceedings of the Healthcare. MDPI, 2021, Vol. 9, p. 137.
2. Welzel, C.; Ostermann, M.; Smith, H.L.; Minssen, T.; Kirsten, T.; Gilbert, S. Enabling secure and self determined health data sharing and consent management. *npj Digital Medicine* **2025**, *8*, 560.
3. Li, K.; Sai, A.R.; Urovi, V. Do you need a blockchain in healthcare data sharing? A tertiary review. *Exploration of Digital Health Technologies* **2024**, *2*, 101–123.
4. Béres, F.; Seres, I.A.; Benczúr, A.A.; Quintyne-Collins, M. Blockchain is watching you: Profiling and deanonymizing ethereum users. In Proceedings of the 2021 IEEE international conference on decentralized applications and infrastructures (DAPPS). IEEE, 2021, pp. 69–78.
5. Li, K.; Lohachab, A.; Dumontier, M.; Urovi, V. Privacy preservation in blockchain-based healthcare data sharing: A systematic review. *Peer-to-Peer Networking and Applications* **2025**, *18*, 1–53.
6. Lawson, J.; Cabili, M.N.; Kerry, G.; Boughtwood, T.; Thorogood, A.; Alper, P.; Bowers, S.R.; Boyles, R.R.; Brookes, A.J.; Brush, M.; et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics* **2021**, *1*.
7. Rondelet, A.; Zajac, M. Zeth: On integrating zerocash on ethereum. *arXiv preprint arXiv:1904.00905* **2019**.
8. Williamson, Z.J. The aztec protocol. *URL: https://github. com/AztecProtocol/AZTEC* **2018**.
9. Nadler, M.; Schär, F. Tornado cash and blockchain privacy: a primer for economists and policymakers. *Federal Reserve Bank of St. Louis Review* **2023**.
10. Ibraimi, L.; Petkovic, M.; Nikova, S.; Hartel, P.; Jonker, W. Ciphertext-policy attribute-based threshold decryption with flexible delegation and revocation of user attributes. *IEEE Trans. Image process* **2009**.
11. Ullah, S.S.; Oleshchuk, V.; Pussewalage, H.S.G. A survey on blockchain envisioned attribute based access control for internet of things: Overview, comparative analysis, and open research challenges. *Computer Networks* **2023**, *235*, 109994.
12. Zhou, L.; Diro, A.; Saini, A.; Kaisar, S.; Hiep, P.C. Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities. *Journal of Information Security and Applications* **2024**, *80*, 103678.
13. Hu, R.; Ma, Z.; Li, L.; Zuo, P.; Li, X.; Wei, J.; Liu, S. An access control scheme based on blockchain and ciphertext policy-attribute based encryption. *Sensors* **2023**, *23*, 8038.
14. World Health Organization. International Statistical Classification of Diseases and Related Health Problems (10th Revision), 2019. Accessed: 2026-07-08.
15. Dyke, S.O.; Philippakis, A.A.; Rambla De Argila, J.; Paltoo, D.N.; Luetkemeier, E.S.; Knoppers, B.M.; Brookes, A.J.; Spalding, J.D.; Thompson, M.; Roos, M.; et al. Consent codes: upholding standard data use conditions. *PLoS genetics* **2016**, *12*, e1005772.
16. Bethencourt, J.; Sahai, A.; Waters, B. Ciphertext-policy attribute-based encryption. In Proceedings of the 2007 IEEE symposium on security and privacy (SP'07). IEEE, 2007, pp. 321–334.

17. Ateniese, G.; Fu, K.; Green, M.; Hohenberger, S. Improved proxy re-encryption schemes with applications to secure distributed storage. *ACM Transactions on Information and System Security (TISSEC)* **2006**, *9*, 1–30.
18. Groth, J. On the Size of Pairing-Based Non-interactive Arguments. In Proceedings of the Advances in Cryptology – EUROCRYPT 2016. Springer, 2016, pp. 305–326.
19. Sasson, E.B.; Chiesa, A.; Garman, C.; Green, M.; Miers, I.; Tromer, E.; Virza, M. Zerocash: Decentralized anonymous payments from bitcoin. In Proceedings of the 2014 IEEE symposium on security and privacy. IEEE, 2014, pp. 459–474.
20. Wu, N.; Xu, L.; Zhu, L. A blockchain based access control scheme with hidden policy and attribute. *Future Generation Computer Systems* **2023**, *141*, 186–196.
21. Boneh, D.; Sahai, A.; Waters, B. Functional encryption: Definitions and challenges. In Proceedings of the Theory of Cryptography Conference. Springer, 2011, pp. 253–273.
22. Secret Network. Secret Network Overview. https://docs.scrt.network/secret-network-documentation/overview-ecosystem-and-technology/secret-network-overview. Accessed: 2026-06-29.
23. Cheng, R.; Zhang, F.; Kos, J.; He, W.; Hynes, N.; Johnson, N.; Juels, A.; Miller, A.; Song, D. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In Proceedings of the 2019 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, 2019, pp. 185–200.
24. Ardagna, C.A.; De Capitani di Vimercati, S.; Paraboschi, S.; Pedrini, E.; Samarati, P. An XACML-based privacy-centered access control system. In Proceedings of the Proceedings of the first ACM workshop on Information security governance, 2009, pp. 49–58.
25. Sheikhalishahi, M.; Tillem, G.; Erkin, Z.; Zannone, N. Privacy-preserving multi-party access control. In Proceedings of the Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society, 2019, pp. 1–13.
26. Gabizon, A.; Williamson, Z.J.; Ciobotaru, O. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* **2019**.
27. Campanelli, M.; Fiore, D.; Querol, A. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In Proceedings of the Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 2075–2092.
28. Liu, T.; Zhang, Z.; Zhang, Y.; Hu, W.; Zhang, Y. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. *Journal of Cryptology* **2025**, *38*, 17.
29. Miller, T.; Alvarez, B.; Hoang, T. VTBC: Privatizing the Volume and Timing of Transactions for Blockchain Applications. In Proceedings of the 2023 32nd International Conference on Computer Communications and Networks (ICCCN). IEEE, 2023, pp. 1–10.
30. Bandrowski, A.; Brinkman, R.; Brochhausen, M.; Brush, M.H.; Bug, B.; Chibucos, M.C.; Clancy, K.; Courtot, M.; Derom, D.; Dumontier, M.; et al. The Ontology for Biomedical Investigations. *PLoS ONE* **2016**, *11*, e0154556.
31. Jaccard, P. The Distribution of the Flora in the Alpine Zone. *New Phytologist* **1912**, *11*, 37–50.


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.